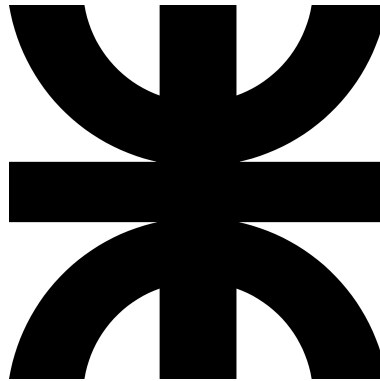


Universidad Tecnológica Nacional

Facultad Regional Córdoba



Carrera de Ingeniería en Electrónica

TÉCNICAS DIGITALES III

Profesor titular: Ing. Gutierrez, Guillermo.

Jefe TP: Ing. Benasulin, Dimas.

TRABAJO PRÁCTICO N°3

“Instrumento virtual basado en ESP32”

INTEGRANTES:

Barbará, Darío

Campos, Luciano

Wu, Abel

Leg : 48911

Leg. : 70745

Leg. : 87023

Resumen

El presente informe corresponde al práctico N°3 de la cátedra de “Técnicas Digitales III” de la Universidad Tecnológica Nacional, Facultad Regional Córdoba.

El documento presenta el registro del diseño, implementación y puesta en marcha de un instrumento virtual basado en un microcontrolador ESP32 y una unidad de medición inercial MPU6050.

El instrumento cuenta con cuatro entradas analógicas (ADC) de 12 bits de resolución, una salida analógica (DAC) de 8 bits de resolución, una entrada digital utilizada para el control de los datos transmitidos y una interfaz de comunicación UART para el envío de datos a una PC.

El informe se estructura en seis secciones principales y cuatro anexos.

En la primera sección se presentan las especificaciones de diseño y la arquitectura general del sistema, tanto a nivel de hardware como de software.

La segunda sección aborda el diseño del soporte de hardware, describiendo los módulos empleados y las conexiones entre ellos. En la tercera sección se desarrolla la arquitectura y el diseño del firmware, implementado sobre el sistema operativo FreeRTOS, junto con la descripción de las tareas y periféricos utilizados.

La cuarta sección corresponde a la verificación de los requisitos de diseño, mientras que la quinta presenta la validación experimental del sistema y el análisis de resultados.

Finalmente, la quinta sección expone las conclusiones generales del trabajo.

Los anexos incluyen el código fuente de las tareas principales del sistema y de la aplicación de interfaz gráfica desarrollada en PyQt5.

Todo el material expuesto en este informe puede consultarse en el siguiente repositorio: [TDIII₂025](#)

Contents

1	Requisitos de diseño	3
1.1	Requisitos funcionales	3
1.2	Requisitos técnicos	3
2	Hardware	4
3	Firmware	5
3.1	Requisitos de velocidad	5
3.1.1	Trama de salida	5
3.1.2	Frecuencia de muestreo de entradas analógicas	6
3.2	Manejo de Periféricos	7
3.3	Modo DEBUG	8
3.4	Interrupcción externa	9
3.5	FreeRTOS	9
3.6	FreeRTOS: Cola xQueueUART	10
3.7	FreeRTOS: Tarea de lectura de canales analógicos	10
3.7.1	Filtrado digital	11
3.8	Filtro digital FIR	11
3.9	FreeRTOS: Tarea de lectura del sensor MPU6050	12
3.10	FreeRTOS: Tarea de Comunicación a PC	12
4	Interfaz de usuario	13
5	Conclusiones	15
6	Anexo A: Implementación de la tarea vTaskManejoCanalesAnalogicos	16
7	Anexo B: Implementación de la tarea vTaskLecturaMPU6050	20
8	Anexo C: Implementación de la tarea vTaskComunicacionUart	21
9	Anexo D: Interfaz de usuario PyQT	25

1 Requisitos de diseño

A continuación se dividen los requisitos del sistema en funcionales y técnicos.

1.1 Requisitos funcionales

Debajo, en la tabla 1, se listan los requisitos funcionales definidos para el trabajo. Estos requisitos determinan la interacción entre el microcontrolador, el sensor inercial y la aplicación de interfaz gráfica, así como las condiciones de entrada y salida de información.

La tabla debajo, tabla 1, resume los principales requisitos definidos en esta etapa de diseño.

Item	Detalle
RF-01	El sistema debe poseer 4 (cuatro) entradas analógicas
RF-02	El sistema debe poseer 1 (una) salida analógica generada a partir de al menos 1 (una) de las entradas analógicas
RF-03	El sistema debe interactuar con un sensor MPU6050 mediante i2c para obtener valores de aceleración y giroscopio
RF-04	El sistema debe comunicar el estado de sus canales analógicos y del sensor MPU6050 a una PC
RF-05	El sistema debe poseer 1 (uno) pulsador para seleccionar la información a enviar al PC

Table 1: Requisitos funcionales del sistema.

1.2 Requisitos técnicos

Se detallan a continuación, tabla 2, los requisitos técnicos para el proyecto derivados de los requisitos funcionales anteriores.

Item	Detalle
RT-01	La plataforma deberá estar programada en FreeRTOS.
RT-02	El sistema debe realizar la lectura de las entradas analógicas de forma periódica
RT-03	La señal DAC debe sintetizarse a partir de las entradas analógicas del sistema.
RT-04	La señal del pulsador debe ser atendida como una interrupción externa.
RT-05	La señal de interrupción interna del pulsador debe notificar a una tarea para cambiar los datos a enviar
RT-06	Existirán tres modos, seleccionables mediante el pulsador, de envío de datos: 1. Estado de los canales analógicos (entradas y salida). 2. Lecturas del sensor MPU6050. 3. Estados de los canales analógicos y Lecturas del sensor MPU6050.
RT-07	La comunicación con la PC se hará mediante puerto serie a 115200 baudios.

Table 2: Requisitos Técnicos del sistema

2 Hardware

El sistema se implementó utilizando módulos comerciales siguiendo la conexión indicada en la figura 1.

El núcleo del diseño está constituido por una placa ESP32-WROOM-32D, basada en un microcontrolador de doble núcleo Xtensa LX6 que opera a 240 MHz, sobre la cual se ejecuta el firmware desarrollado en FreeRTOS.

La comunicación con el sensor inercial MPU6050 se realiza mediante el bus I²C, configurado a una frecuencia de 400 kHz, lo que permite la adquisición continua de los valores de aceleración y velocidad angular en los tres ejes.

El sistema cuenta con un pulsador de entrada configurado con lógica negativa (activo en bajo), asociado a un circuito antirrebote pasivo simple y a una interrupción externa que permite seleccionar el modo de transmisión de datos.

Para los canales analógicos se implementaron filtros antialiasing de primer orden con una frecuencia de corte $f_c 4.35[kHz]$ y protección contra sobretensión e inversión de polaridad.

La alimentación del sistema se toma del puerto USB de la placa ESP32, que suministra la energía necesaria tanto para el microcontrolador como para el sensor MPU6050 y los circuitos asociados.

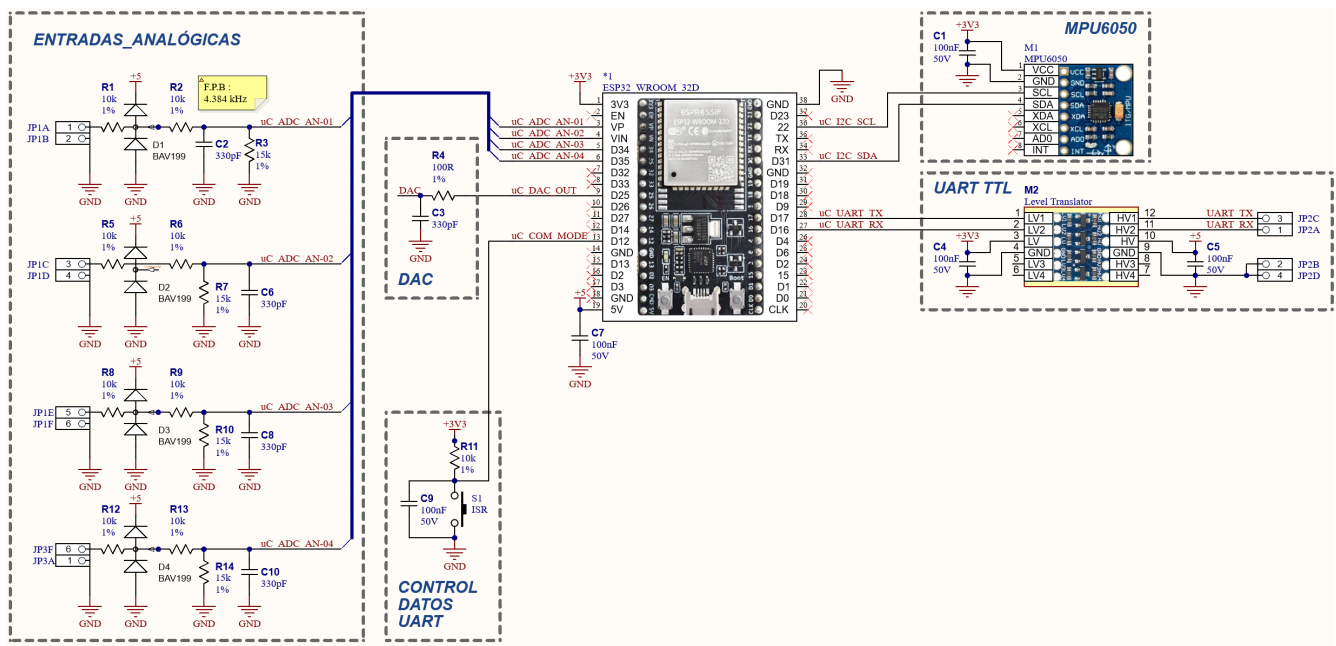


Figure 1: Circuito esquemático implementado

3 Firmware

3.1 Requisitos de velocidad

3.1.1 Trama de salida

El mensaje de salida está formado por : un encabezado para identificación del contenido del mensaje, el estado de las entradas y salidas analógicas, los datos del acelerómetro (x, y, z), los datos del giroscopio (x,y,z), un método de comprobación de errores CRC-16. El detalle de los elementos que componen el mensaje de salida del instrumento a la PC se muestran en la tabla debajo (tabla 3). Se

Bloque	Elemento	Resolución	Total
Comunicación	Encabezado	8 bits	8 bits
Canales analógicos	Entrada analógica 01	16 bits	64 bits
	Entrada analógica 02	16 bits	
	Entrada analógica 03	16 bits	
	Entrada analógica 04	16 bits	
	Salida analógica	8 bits	8 bits
Acelerómetro MPU6050	Aceleración eje x	16 bits	48 bits
	Aceleración eje y	16 bits	
	Aceleración eje z	16 bits	
Giroscopio MPU6050	Giroscopio eje x	16 bits	48 bits
	Giroscopio eje y	16 bits	
	Giroscopio eje z	16 bits	
Comunicación	CRC-16	16 bits	16 bits
		192 bits	192 bits

Table 3: Parámetros de la trama de salida

La trama del mensaje de salida se define como se muestra en la tabla 4 a continuación :

Byte	Elemento
00	Encabezado
01	Entrada Analógica 01 (H)
02	Entrada Analógica 01 (L)
03	Entrada Analógica 02 (H)
04	Entrada Analógica 02 (L)
05	Entrada Analógica 03 (H)
06	Entrada Analógica 03 (L)
07	Entrada Analógica 04 (H)
08	Entrada Analógica 04 (L)
09	Salida Analógica
10	Acelerómetro x (H)
11	Acelerómetro x (L)
12	Acelerómetro y (H)
13	Acelerómetro y (L)
14	Acelerómetro z (H)
15	Acelerómetro z (L)
16	Giroscopio x (H)
17	Giroscopio x (L)
18	Giroscopio y (H)
19	Giroscopio y (L)
20	Giroscopio z (H)
21	Giroscopio z (L)
22	CRC-16 (H)
23	CRC-16 (L)

Table 4: Trama del mensaje de salida.

3.1.2 Frecuencia de muestreo de entradas analógicas

Dada la especificación de la velocidad de comunicación $B.R.$, la longitud máxima del mensaje de salida hacia la PC N_{Trama} , la longitud de la respuesta de la pc al sistema N_{ACK} , y el porcentaje de canal libre deseado K , se puede calcular la máxima frecuencia de muestreo útil para los conversores A/D de las entradas analógicas (y por ende el ancho de banda de las entradas analógicas) como se muestra a continuación.

El tiempo mínimo de comunicación (transmisión y recepción) $t_{c(min)}$ se calcula mediante la cantidad total de datos y la velocidad de comunicación como:

$$t_{c(min)} = \frac{N_{Trama} + N_{ACK}}{B.R.} \quad (1)$$

Donde $1/B.R$ es el tiempo de bit. Considerando el porcentaje de canal libre que se desea en la comunicación, la ecuación anterior se modifica como:

$$t_c = \frac{N_{Trama} + N_{ACK}}{(1 - k) \cdot B.R.}; \quad 0 \leq k < 1 \quad (2)$$

Para que la señal reconstruida en el extremo del PC no tenga aliasing es necesario que no se pierdan muestras, por

lo tanto, el período mínimo de muestreo será igual a t_c y la frecuencia de muestreo $f_{s(max)}$ resulta:

$$f_{s(max)} = \frac{1}{t_c} = \frac{(1-k) \cdot B.R.}{N_{Trama} + N_{ACK}} \quad (3)$$

Por último, el ancho de banda útil de las entradas analógicas resulta

$$BW_{ent} \leq \frac{f_{s(max)}}{2} = \frac{1}{2} \cdot \frac{(1-k) \cdot B.R.}{N_{Trama} + N_{ACK}} \quad (4)$$

En base a las especificaciones de proyecto se tiene:

- Baud Rate : 115200 baudios.
- Longitud del mensaje de salida: 192 bits, 24 bytes.
- Longitud del mensaje de respuesta: 8 bits, 1 byte.
- Codificación del mensaje 8N1.
- Porcentaje de canal libre: 25% ($k = 0.25$).

La máxima frecuencia de muestreo se calcula mediante **3** como

$$f_{s(max)} = \frac{(1 - 0.25) \cdot 115200 \left[\frac{\text{simbolo}}{\text{seg}} \right]}{(24[\text{simbolo}] + 1[\text{simbolo}]) \cdot 10 \left[\frac{\text{bits}}{\text{simbolo}} \right]} = 345.6[Hz] \quad (5)$$

El ancho de banda útil del canal analógico resulta entonces

$$BW_{ent} = \frac{f_s}{2} = \frac{345.6[Hz]}{2} = 172.8[Hz] \quad (6)$$

Se adopta una frecuencia efectiva de 100 [Hz].

3.2 Manejo de Periféricos

Los periféricos del sistema (ADC, DAC, UART y MPU6050) se inicializan mediante las funciones especializadas: **eInicializarPeriféricos** y **eInicializarMPU6050**.

Ambas son invocadas desde la función principal (app_main) y se encargan de verificar la correcta configuración de cada dispositivo, proporcionando información detallada del proceso cuando el sistema se ejecuta en modo debug.

Debajo se adjunta la implementación de la función **eInicializarMPU6050**, la cual inicia el driver i2C y el sensor MPU6050 a través de la librería *mpu6050.h*. En modo debug la función emite por el puerto serie el estado de cada paso de inicialización.


```
1      esp_err_t eInicializarMPU6050(void)
2      {
3          if (DEBUG_MODE)
4              ESP_LOGI(TAG, "[START]\nteInicializarMPU6050 start");
5
6          esp_err_t resultado;
7
8          i2c_config_t conf = {
9              .mode = I2C_MODE_MASTER,
10             .sda_io_num = I2C_MASTER_SDA_IO,
11             .sda_pullup_en = GPIO_PULLUP_ENABLE,
12             .scl_io_num = I2C_MASTER_SCL_IO,
13             .scl_pullup_en = GPIO_PULLUP_ENABLE,
14             .master.clk_speed = I2C_MASTER_FREQ_HZ,
15         };
16
17         resultado = i2c_param_config(I2C_MASTER_NUM, &conf);
18         if (resultado != ESP_OK && DEBUG_MODE)
19         {
20             printf("[FALLA]\tI2C error de configuración.");
21             return resultado;
22         }
23
24         resultado = i2c_driver_install(I2C_MASTER_NUM, conf.mode,
25             0, 0, ESP_INTR_FLAG_DEFAULT);
26         if (resultado != ESP_OK && DEBUG_MODE)
27         {
28             printf("[FALLA]\tI2C error de configuración.");
29             return resultado;
30         }
31
32         resultado = mpu6050_init(I2C_MASTER_NUM);
33         if (resultado != ESP_OK && DEBUG_MODE)
34         {
35             printf(TAG, "[FALLA]\tError al inicializar la
36                 comunicación I2C con el MPU6050");
37             return resultado;
38         }
39
40         if (DEBUG_MODE)
41             ESP_LOGI(TAG, "[OK]\nteInicializarMPU6050 OK");
42
43         return resultado;
44     }
```

Listing 1: Ejemplo de inicialización de perifericos : sensor MPU6050

3.3 Modo DEBUG

El firmware incorpora un modo **DEBUG** controlado por una bandera global bajo el nombre de **MODULO_DEBUG**. Este modo utiliza la biblioteca *esp_log.h* para enviar mensajes de diagnóstico a través del puerto serie asociado al conector USB de la placa ESP32, mostrando el estado de ejecución de las distintas secciones del código.

3.4 Interrupción externa

La interrupción externa asociada al pulsador se gestiona mediante dos funciones: **eInicializarISR_Pulsador** y **vISR_Handler_Pulsador**.

La primera configura el servicio de interrupción, definiendo el flanco de bajada como evento de disparo, dado que el pulsador trabaja con lógica inversa (activo en bajo).

La segunda función atiende la interrupción propiamente dicha, implementando un mecanismo de anti-rebote por software basado en el tiempo transcurrido entre activaciones, y notifica a la tarea de comunicación UART para que actualice la cadena de salida.

Debajo se muestra la implementación de la función **vISR_Handler_Pulsador**.

```
1         void IRAM_ATTR vISR_Handler_Pulsador(void *arg)
2         {
3             BaseType_t xHigherPriorityTaskWoken = pdFALSE;
4             static TickType_t xTickPrevio = 0;           // tick
5                 FreeRTOS del último pulso. Antirebote
6             const TickType_t xTickActual = xTaskGetTickCount(); // tick
7                 FreeRTOS actual. Antirebote
8
9             // Antirebote simple
10            if ((xTickActual - xTickPrevio) >= pdMS_TO_TICKS(
11                DEBOUNCE_MS))
12            {
13                xTickPrevio = xTickActual;
14                xTaskNotifyFromISR(xHandleComUART, UART_NOTIFY_TOGGLE,
15                    eSetBits, &xHigherPriorityTaskWoken);
16                if (xHigherPriorityTaskWoken)
17                {
18                    portYIELD_FROM_ISR();
19                }
20            }
21        }
```

Listing 2: Implementación de la función **vISR_Handler_Pulsador**

3.5 FreeRTOS

El sistema se ejecuta sobre FreeRTOS y está estructurado en tres tareas principales:

1. **vTaskManejoCanalesAnalogicos**: Encargada de la lectura, escritura y publicación del estado de las entradas analógicas.
2. **vTaskLecturaMPU6050**: Encargada de la lectura y publicación de los datos del sensor MPU6050.
3. **vTaskComunicacionUart**: Encargada del envío de datos por puerto serie.

Las tareas comparten información mediante una cola de tamaño 1 denominada **xQueueUART** y utilizan un semáforo **xMutexCola** para evitar condiciones de carrera durante el acceso concurrente.

3.6 FreeRTOS: Cola xQueueUART

La cola emplea la estructura mostrada en el Listado (3), que contiene un arreglo de 16 bits para los cuatro canales ADC, un valor de salida DAC, y los datos crudos de aceleración y giro provenientes del MPU6050.

La cola se define con tamaño 1 ya que su función es mantener únicamente el último conjunto de datos válido disponible para transmisión, evitando la acumulación de muestras obsoletas. Esta configuración reduce la latencia, simplifica la lógica de sincronización entre tareas y optimiza el uso de memoria dinámica en el sistema FreeRTOS.

```
1         typedef struct
2         {
3             uint8_t valor_DAC;           // Modo de transmisión
4             uint16_t Valor_ADC[4];       // Array para los 4
5             uint16_t iAceleracion_raw[3]; // Aceleración raw del
6             uint16_t iGiro_raw[3];       // Giro raw del MPU6050
7         } structColaFreeRTOS;
```

Listing 3: Estructura de la cola xQueueUART

3.7 FreeRTOS: Tarea de lectura de canales analógicos

La tarea **vTaskManejoCanalesAnalogicos** se encarga de la adquisición, filtrado y publicación del estado de las entradas analógicas del sistema, así como de la generación de la señal de salida analógica mediante el DAC interno del ESP32.

Las entradas analógicas se definieron en las GPIO 34, 35, 36 Y 39 asociados al módulo ADC1 del microcontrolador. La lectura de los canales analógicos se realiza en modo continuo a 10 (diez) kHz por canal (40 kHz en total).

Cada grupo de muestras recibido por DMA se procesa canal por canal mediante un filtro FIR de 4 coeficientes, con una frecuencia de corte aproximada de 200 Hz. Posteriormente, los valores filtrados se acumulan y promedian para reducir ruido y realizar un diezmado que fija la tasa de actualización en 100 Hz, acorde a la frecuencia de publicación de datos por UART.

A partir de los valores analógicos promediados, la tarea calcula en cada ciclo los parámetros de amplitud, desplazamiento (offset) y pendiente de una señal triangular generada por el DAC del microcontrolador. Esta señal se actualiza a 100 Hz, modulando su forma de acuerdo con las entradas analógicas.

Una vez actualizados los valores de los canales ADC y del DAC, la tarea los almacena en la estructura **structColaFreeRTOS** y los publica en la cola **xQueueUART** bajo protección del semáforo **xMutexCola**, permitiendo que las demás tareas accedan de forma sincronizada a los datos más recientes.

En conjunto, esta tarea implementa un proceso completo de adquisición, procesamiento digital y generación analógica, operando con temporización precisa y sin interrupciones explícitas, gracias al uso del ADC continuo con DMA y la planificación determinista de FreeRTOS.

La implementación de esta tarea puede consultarse en el anexo A del informe.

3.7.1 Filtrado digital

3.8 Filtro digital FIR

En reemplazo de un filtro antialiasing de entrada el sistema implementa un **filtro digital FIR (Finite Impulse Response)** de orden 3 (cuatro coeficientes) en la tarea `vTaskManejoCanalesAnalogicos` para filtrar el ruido presente en los canales analógicos.

El filtro se implementó mediante la ecuación de convolución discreta:

$$y[n] = h_0 x[n] + h_1 x[n-1] + h_2 x[n-2] + h_3 x[n-3] \quad (7)$$

Donde $x[n]$ representa las muestras de entrada y h_k los coeficientes del filtro.

Dado el ancho de banda deseado para las entradas analógicas ($BW = 100[Hz]$), el filtro se diseñó con una frecuencia de corte de $f_c = 200[Hz]$ a una frecuencia de muestreo $f_s = 10[kHz]$; para este filtros los coeficientes obtenidos son los mostrados a continuación:

$$h = [0.0468 \quad 0.4532 \quad 0.4532 \quad 0.0468] \quad (8)$$

El procesamiento se realiza en tiempo real dentro del bucle principal de adquisición, inmediatamente después de recibir un bloque de datos desde el módulo ADC en modo continuo con DMA.

Adicionalmente el filtro se complementó con un diezmado que genera da como resultado frecuencia efectiva de $f_e = 100[Hz]$. La respuesta del filtro se muestra debajo en la figura 2.

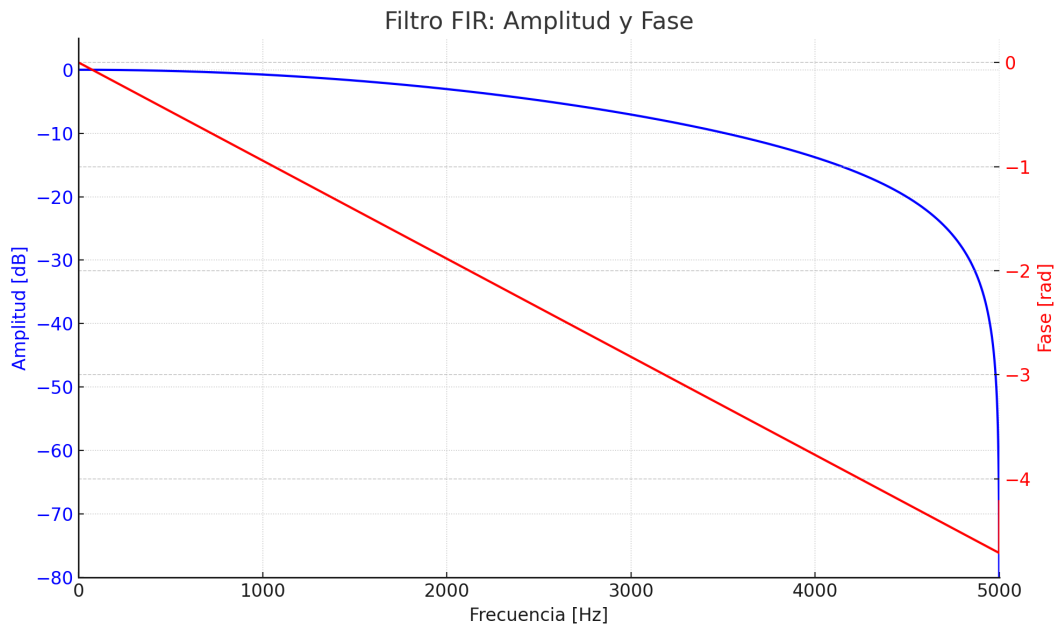


Figure 2: Respuesta de amplitud y fase del filtro FIR implementado.

En conjunto, la aplicación del filtro FIR permite mejorar la relación señal-ruido de las mediciones analógicas, reemplazando la necesidad de un filtro antialiasing analógico externo y demostrando la eficacia del procesamiento

digital en el dominio del tiempo.

3.9 FreeRTOS: Tarea de lectura del sensor MPU6050

La tarea **vTaskLecturaMPU6050** realiza la adquisición periódica de los datos crudos del sensor inercial MPU6050 mediante el bus I²C y publica esos valores en la cola compartida del sistema.

Al inicio, ejecuta una rutina de calibración del acelerómetro y del giroscopio y realiza la inicialización del filtro de roll/pitch. En su bucle principal, toma el mutex de acceso a la cola, lee los registros crudos de aceleración y velocidad angular (ejes X, Y, Z) y sobrescribe la estructura compartida en xQueueUART usando xQueueOverwrite, para que siempre quede disponible el último conjunto de datos válido. Finalmente libera el mutex y entra en espera 100 ms para fijar una cadencia de muestreo aproximada de 100 Hz.

Esta tarea no convierte a unidades físicas ni calcula ángulos; solo publica datos crudos para que la tarea UART realice la conversión, fusión sensorial y reporte según el modo activo. El uso combinado del mutex y la cola de tamaño 1 garantiza coherencia de datos y baja latencia en la comunicación entre tareas.

La implementación de esta tarea puede consultarse en el anexo B del informe.

3.10 FreeRTOS: Tarea de Comunicación a PC

La tarea **vTaskComunicacionUart** tiene como función principal el formateo y transmisión de los datos adquiridos por el sistema a través del puerto serie UART2, permitiendo la comunicación con la aplicación de interfaz en PC.

Su ejecución se organiza en torno a un esquema de notificaciones de tarea, donde cada evento relevante (nuevo conjunto de datos o cambio de modo de transmisión) activa el procesamiento correspondiente. En particular:

El bit *UART_NOTIFY_TOGGLE*, generado por la interrupción externa del pulsador, provoca el cambio de modo de transmisión.

El bit *UART_NOTIFY_WAKE*, emitido por las tareas de adquisición, indica la disponibilidad de nuevos datos en la cola xQueueUART.

Al recibir una notificación, la tarea extrae la última estructura publicada en la cola (protegiendo el acceso con el semáforo xMutexCola) y construye el mensaje UART de acuerdo con el modo de transmisión actual:

- Modo ADC–DAC: envía las cuatro lecturas analógicas y el valor del DAC.
- Modo MPU: envía los valores crudos de aceleración y giro del sensor MPU6050.
- Modo combinado (ADC–DAC + MPU): transmite simultáneamente los datos analógicos, el valor del DAC y las lecturas del sensor inercial.

En cada caso, la trama transmitida comienza con un byte de encabezado que identifica el modo operativo y concluye con un CRC de 16 bits calculado mediante la función interna *crc_uart()*.

Esta arquitectura garantiza una comunicación determinista y sin bloqueos, ya que la tarea UART no espera acumulación de mensajes, sino que siempre transmite el último conjunto de datos válido disponible.

En modo debug, la tarea imprime por consola el contenido de las variables transmitidas y los ángulos roll y pitch calculados, facilitando la verificación y depuración del sistema en tiempo real.

La implementación de esta tarea puede consultarse en el anexo C del informe.

4 Interfaz de usuario

Se desarrolló una aplicación de interfaz gráfica en Python utilizando el framework PyQt5 y la biblioteca PyQtGraph para la representación en tiempo real de las señales adquiridas por el sistema embebido.

El programa establece una comunicación serie mediante la clase QSerialPort, configurando el puerto, velocidad y control de flujo de acuerdo con los parámetros del firmware (115200 bps, 8N1). A partir del encabezado de cada trama recibida, la aplicación interpreta el modo de funcionamiento actual y decodifica los datos correspondientes:

- Modo ADC–DAC: se grafican en tiempo real las cuatro señales analógicas de entrada y el valor de salida del DAC.
- Modo MPU: se visualizan los datos crudos de aceleración y velocidad angular provenientes del sensor MPU6050, junto con el cálculo de los ángulos de roll y pitch.
- Modo combinado (Full): se muestran de manera simultánea las señales analógicas, el valor del DAC y las lecturas inerciales del MPU6050.

La interfaz gráfica permite seleccionar el puerto serie deseado, iniciar o detener la comunicación y cambiar dinámicamente entre modos de visualización. Los gráficos se actualizan mediante temporizadores de alta resolución y estructuras de tipo deque, que permiten mantener una ventana temporal de datos continua y eficiente.

El sistema de visualización está diseñado para que los gráficos mantengan su desplazamiento temporal incluso cuando algún canal no recibe datos (por cambio de modo), preservando así la coherencia temporal del eje de tiempo y la continuidad de la animación.

En la figura debajo, [3](#), se muestra la ventana principal de la interfaz de usuario. La implementación del código se muestra en el anexo D.

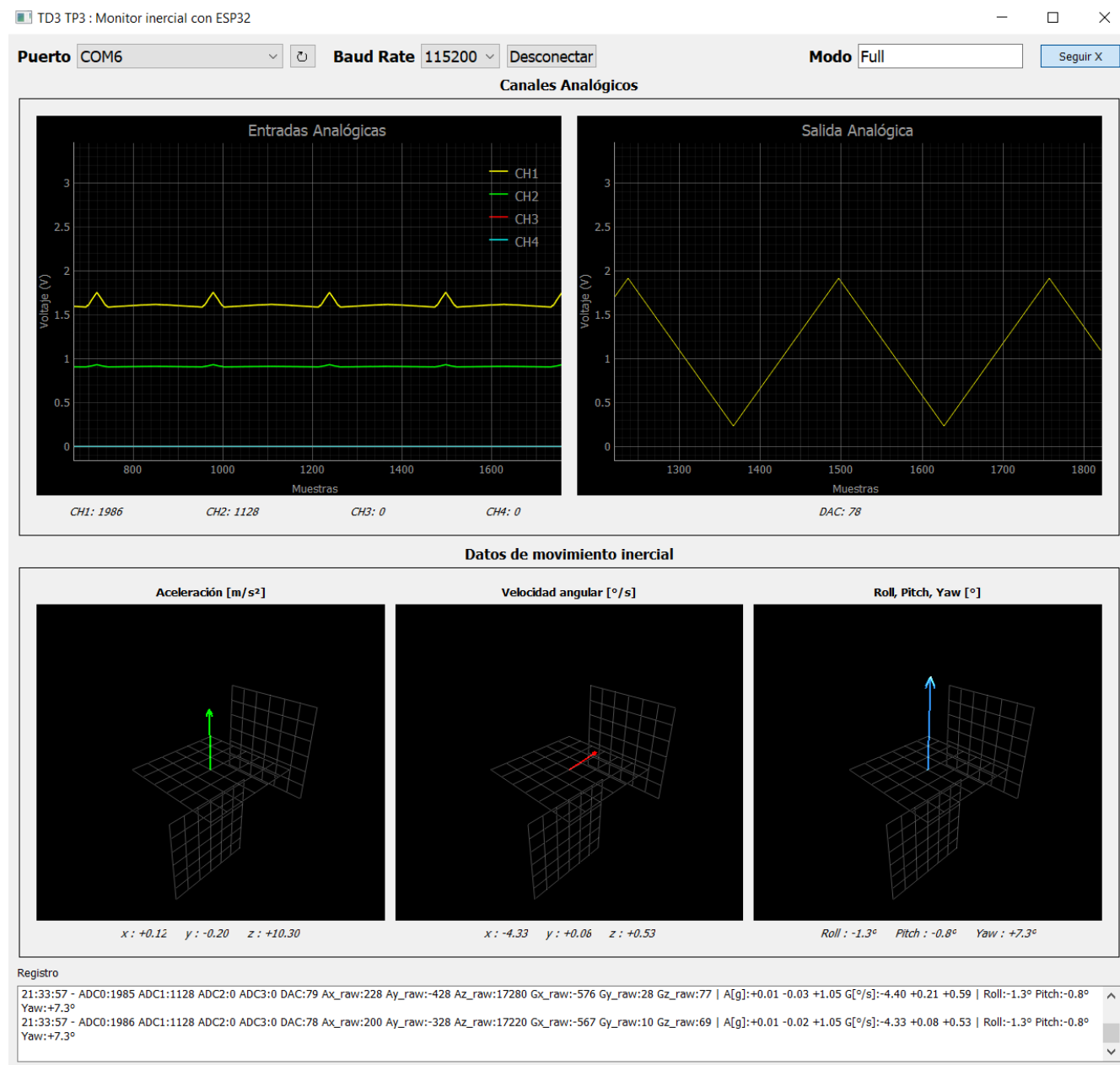


Figure 3: Ventana principal de la interfaz de usuario.

5 Conclusiones

El sistema desarrollado cumple con los *requisitos funcionales y técnicos* establecidos: adquisición de cuatro canales analógicos, generación de una salida analógica (DAC), lectura del sensor inercial MPU6050 por I²C y comunicación con PC por UART a 115200 baudios, con selección de datos de salida mediante pulsador.

En términos de arquitectura, la partición en tareas de **FreeRTOS** resultó adecuada para garantizar ejecución determinista y baja latencia: (i) manejo de canales analógicos, (ii) lectura del MPU6050, y (iii) comunicación UART.

Se verificó que el uso de una **cola de tamaño 1** (uno) y notificaciones de tarea aseguró que la PC recibiera siempre el *último estado válido*, evitando acumulación de datos obsoletos.

Se verificó que el procesamiento digital planteado para las señales pertenecientes a las entradas analógicas aportó atenuación significativa del contenido de alta frecuencia y preservación de la banda útil (hasta el orden de centenas de Hz) con fase prácticamente lineal en banda pasante. Por su parte el promediado de 100 muestras actúa como un filtro de ventana rectangular, aportando una envolvente tipo *sinc* que incrementa la atenuación fuera de banda y reduce variabilidad antes del envío por UART.

Se verificó que, aun empleando filtros antialiasing analógicos de gran ancho de banda para la aplicación (44kHz vs 100Hz), la respuesta de los canales analógicos (evaluada mediante la representación en pantalla del IDE) resultaron estables y coherentes con el bode equivalente total de la cadena DSP implementada.

La interrupción externa del pulsador (lógica inversa, flanco de bajada) y el modo debug basado en `esp_log.h` facilitaron la validación funcional y el trazado de estados internos sin penalizar la operación nominal.

La **aplicación PyQt5** cumplió su objetivo como interfaz de supervisión en tiempo real, con gráficos que mantienen la continuidad temporal, aun ante cambios de modo, y decodificación de tramas con encabezado y CRC-16.

Limitaciones y trabajo futuro

Se identifican como líneas de mejora:

1. Ajuste fino del diseño del FIR (orden y frecuencia de corte) para optimizar la transición y la atenuación en stopband.
2. Parametrización en tiempo de ejecución (por UART) de la tasa de publicación, coeficientes del filtro y ganancia/offset del DAC.
3. Integración de calibración persistente de sesgos del MPU6050 y, eventualmente, fusión sensorial (complementario o cuaterniones) si se requiere yaw/estabilidad mejorada.

En síntesis, el **instrumento virtual** basado en ESP32 y MPU6050 alcanzó los objetivos planteados, ofreciendo una plataforma estable, reproducible y extensible para prácticas y ensayos de adquisición, filtrado digital y visualización en tiempo real.

6 Anexo A: Implementación de la tarea vTaskManejoCanalesAnalogicos

```

1 void vTaskManejoCanalesAnalogicos(void *pvParameters)
2 {
3     // Tasa de muestreo y publicación a cola
4     #define FS_CH 10000U // 10 kHz por canal
5     #define PUB_HZ 100U  // publicar a 100 Hz
6     #define PUB_DIV (FS_CH / PUB_HZ) // 100 muestras por salida
7
8     structColaFreeRTOS sDatosADC = {0}, sColaPrevia = {0};
9
10    // DAC
11    uint8_t dac_valor = 0;
12    int16_t dac_amplitud = 0, dac_max = 0, dac_min = 0;
13    int16_t dac_offset = 128;
14    int8_t dac_step = 1, dac_step_up = 1, dac_step_down = 1;
15
16    // DMA: 40 kHz total -> 10 kHz/ch
17    static adc_continuous_handle_t s_adc = NULL;
18    const adc_continuous_handle_cfg_t hcfg = {
19        .max_store_buf_size = 4096,
20        .conv_frame_size = 256,
21    };
22    ESP_ERROR_CHECK(adc_continuous_new_handle(&hcfg, &s_adc));
23
24    static adc_digi_pattern_config_t patt[4] = {
25        {.atten = ADC_ATTEN_DB_11, .channel = ADC_CHANNEL_0, .unit = ADC_UNIT_1, .
26         bit_width = SOC_ADC_DIGI_MAX_BITWIDTH}, // GPIO36
27        {.atten = ADC_ATTEN_DB_11, .channel = ADC_CHANNEL_3, .unit = ADC_UNIT_1, .
28         bit_width = SOC_ADC_DIGI_MAX_BITWIDTH}, // GPIO39
29        {.atten = ADC_ATTEN_DB_11, .channel = ADC_CHANNEL_6, .unit = ADC_UNIT_1, .
30         bit_width = SOC_ADC_DIGI_MAX_BITWIDTH}, // GPIO34
31        {.atten = ADC_ATTEN_DB_11, .channel = ADC_CHANNEL_7, .unit = ADC_UNIT_1, .
32         bit_width = SOC_ADC_DIGI_MAX_BITWIDTH}, // GPIO35
33    };
34
35    const adc_continuous_config_t ccfg = {
36        .sample_freq_hz = 40000, // 40 kHz totales -> 10 kHz por canal
37        .conv_mode = ADC_CONV_SINGLE_UNIT_1,
38        .format = ADC_DIGI_OUTPUT_FORMAT_TYPE1,
39        .pattern_num = 4,
40        .adc_pattern = patt,
41    };
42
43    ESP_ERROR_CHECK(adc_continuous_config(s_adc, &ccfg));
44    ESP_ERROR_CHECK(adc_continuous_start(s_adc));
45
46    // Buffers
47    static uint8_t rxbuf[512];
48    uint32_t nbytes = 0;
49
50    // Filtro FIR 3er orden (4 taps) por canal ( f c 200 Hz @ 10 kHz)
51    static const float FIR_H[4] = {

```

```

48         0.046834613683134206f,
49         0.45316538631686565f,
50         0.45316538631686565f,
51         0.046834613683134206f
52     };
53
54     static float xbuf[4][4] = {{0}}; // [canal][tap] = x[n], x[n-1], x[n-2], x[n-3]
55
56     // Diezmado por promedio a 200 Hz
57     static uint32_t acc[4] = {0}; // acumuladores por canal
58     static uint16_t acc_cnt = 0; // cuenta cuántas muestras van en la ventana (0..
        PUB_DIV-1)
59
60     if (DEBUG_MODE)
61         ESP_LOGI(TAG, "[START]\tFreeRTOS: vTaskManejoCanalesAnalogicos (DMA+FIR, %u
            Hz pub)", PUB_HZ);
62
63     for (;;)
64     {
65         nbytes = 0;
66
67         // timeout en MILISEGUNDOS
68         esp_err_t er = adc_continuous_read(s_adc, rxbuf, sizeof(rxbuf), &nbytes, 10
            /* ms */);
69         if (er != ESP_OK || nbytes < sizeof(adc_digi_output_data_t))
70         {
71             // Respiración mínima
72             vTaskDelay(pdMS_TO_TICKS(1));
73             continue;
74         }
75
76         for (size_t i = 0; i + sizeof(adc_digi_output_data_t) <= nbytes; i +=
            sizeof(adc_digi_output_data_t))
77         {
78             const adc_digi_output_data_t *s = (const adc_digi_output_data_t *) &
                rxbuf[i];
79
80             // Mapear canal HW      índice 0..3
81             int k;
82             switch (s->type1.channel)
83             {
84                 case ADC_CHANNEL_0: k = 0; break;
85                 case ADC_CHANNEL_3: k = 1; break;
86                 case ADC_CHANNEL_6: k = 2; break;
87                 case ADC_CHANNEL_7: k = 3; break;
88                 default: continue; // ignora otros
89             }
90
91             // Filtrado
92             xbuf[k][3] = xbuf[k][2];
93             xbuf[k][2] = xbuf[k][1];
94             xbuf[k][1] = xbuf[k][0];
95             xbuf[k][0] = (float)(s->type1.data & 0xFFFF); // 12 bits efectivos
96             float y = FIR_H[0] * xbuf[k][0] + FIR_H[1] * xbuf[k][1] + FIR_H[2] *
                xbuf[k][2] + FIR_H[3] * xbuf[k][3];

```

```

97
98 // Clipeado a 12 bits y almacenamiento del valor filtrado instantáneo
99 int32_t v = (int32_t)(y + 0.5f);
100 if (v < 0) v = 0;
101 else if (v > 4095) v = 4095;
102 sDatosADC.Valor_ADC[k] = (uint16_t)v;
103
104 // Acumulación
105 if (k == 3)
106 {
107     // Acumular promedio simple (anti-ruido + diezmado)
108     for (int ch = 0; ch < 4; ch++)
109     {
110         acc[ch] += sDatosADC.Valor_ADC[ch];
111     }
112     acc_cnt++;
113
114     // Publicación a 100Hz
115     if (acc_cnt >= PUB_DIV)
116     {
117         // Promediar y resetear acumuladores
118         for (int ch = 0; ch < 4; ch++)
119         {
120             uint32_t avg = acc[ch] / PUB_DIV;
121             if (avg > 4095) avg = 4095;
122             sDatosADC.Valor_ADC[ch] = (uint16_t)avg;
123             acc[ch] = 0;
124         }
125         acc_cnt = 0;
126
127         // ===== Generación DAC (triangular) a 100 Hz =====
128         dac_amplitud = (int16_t)(128 * (int32_t)sDatosADC.Valor_ADC
129             [0] / 4096);
130         dac_offset = (int16_t)((255 * (int32_t)sDatosADC.Valor_ADC
131             [1] / 4096) + 10);
132         dac_step_up = (int8_t)((9 * (int32_t)sDatosADC.Valor_ADC[2]
133             / 4096) + 1);
134         dac_step_down = (int8_t)((9 * (int32_t)sDatosADC.Valor_ADC[3]
135             / 4096) + 1);
136
137         dac_max = dac_offset + dac_amplitud;
138         dac_min = dac_offset - dac_amplitud;
139
140         dac_valor = (uint8_t)(dac_valor + dac_step);
141
142         if (dac_valor >= dac_max)
143         {
144             dac_valor = (uint8_t)(dac_max < 0 ? 0 : (dac_max > 255 ?
145                 255 : dac_max));
146             dac_step = -(int8_t)dac_step_down;
147         }
148         else if (dac_valor <= dac_min)
149         {
150             dac_valor = (uint8_t)(dac_min < 0 ? 0 : (dac_min > 255 ?
151                 255 : dac_min));
152         }
153     }
154 }

```

```

146         dac_step = (int8_t)dac_step_up;
147     }
148
149     // Escribir DAC (GPIO25)
150     dac_output_voltage(DAC_CHAN_0, dac_valor);
151     sDatosADC.valor_DAC = dac_valor;
152
153     // Publicar datos a la cola (con mutex)
154     if (xSemaphoreTake(xMutexCola, pdMS_TO_TICKS(1)) == pdTRUE)
155     {
156         if (xQueuePeek(xQueueUART, &sColaPrevia, 0) == pdTRUE)
157         {
158             for (int j = 0; j < 3; j++)
159             {
160                 sDatosADC.iAceleracion_raw[j] = sColaPrevia.
                    iAceleracion_raw[j];
161                 sDatosADC.iGiro_raw[j] = sColaPrevia.iGiro_raw[j];
162             }
163         }
164
165         // Si la cola está vacía, usar Send para generar transición
            0 -> 1.
166         if (uxQueueMessagesWaiting(xQueueUART) == 0){
167             (void)xQueueSend(xQueueUART, &sDatosADC, 0); //
                despierta la tarea UART
168         }
169         else{
170             (void)xQueueOverwrite(xQueueUART, &sDatosADC); //
                refresca el último (sin crecer)
171         }
172
173         xSemaphoreGive(xMutexCola);
174
175         // bit de WAKE
176         if (xHandleComUART) xTaskNotify(xHandleComUART,
            UART_NOTIFY_WAKE, eSetBits);
177     }
178     } // if (acc_cnt >= PUB_DIV)
179     } // if (k == 3)
180 } // for (i ...)
181
182     taskYIELD();
183 } // for (;;)
184 }

```

Listing 4: Tarea vTaskManejoCanalesAnalogicos: Implementación

7 Anexo B: Implementación de la tarea vTaskLecturaMPU6050

```

1   void vTaskLecturaMPU6050(void *pvParameters)
2   {
3       if (DEBUG_MODE)
4           ESP_LOGI(TAG, "[START]\tFreeRTOS: vTaskLecturaMPU6050 started");
5
6       structColaFreeRTOS out = {0}, prev = {0};
7       float accel_bias[3] = {0}, gyro_bias[3] = {0};
8
9       mpu6050_calibrate(I2C_MASTER_NUM, accel_bias, gyro_bias);
10      roll_pitch_init();
11
12      const TickType_t period = pdMS_TO_TICKS(10);    // 100 Hz
13      TickType_t last = xTaskGetTickCount();
14
15      for(;;)
16      {
17          // Leer MPU SIN mutex
18          if (mpu6050_read_raw_data(I2C_MASTER_NUM,
19                                  &out.iAceleracion_raw[0], &out.iAceleracion_raw[1], &out.
20                                  iAceleracion_raw[2],
21                                  &out.iGiro_raw[0],          &out.iGiro_raw[1],          &out.iGiro_raw
22                                  [2]) == ESP_OK)
23          {
24              if (xSemaphoreTake(xMutexCola, pdMS_TO_TICKS(1)) == pdTRUE)
25              {
26                  // leer últimos ADC/DAC y luego publicar TODO dentro del mismo
27                  // mutex
28                  if (xQueuePeek(xQueueUART, &prev, 0) == pdTRUE) {
29                      out.valor_DAC = prev.valor_DAC;
30                      for (int i=0; i<4; i++) out.Valor_ADC[i] = prev.Valor_ADC[i];
31                  }
32
33                  if (uxQueueMessagesWaiting(xQueueUART) == 0)
34                      (void)xQueueSend(xQueueUART, &out, 0);
35                  else
36                      (void)xQueueOverwrite(xQueueUART, &out);
37
38                  xSemaphoreGive(xMutexCola);
39              }
40          }
41          else if (DEBUG_MODE) {
42              ESP_LOGE(TAG, "[FALLA]\tError al leer datos del MPU6050");
43          }
44
45          vTaskDelayUntil(&last, period); // Ritmo exacto 100 Hz
46      }
47  }

```

Listing 5: Tarea vTaskComunicacionUart: Implementación

8 Anexo C: Implementación de la tarea vTaskComunicacionUart

```

1         void vTaskComunicacionUart(void *pvParameters)
2     {
3         structColaFreeRTOS sDatosSalidaUart;
4         uint8_t modo_transmision = 0;
5
6         float fAceleracion_x, fAceleracion_y, fAceleracion_z;
7         float fGiro_x, fGiro_y, fGiro_z;
8
9         uint8_t mensaje_uart[32];
10        uint16_t crc;
11        size_t n = 0;
12
13        if (DEBUG_MODE)
14            ESP_LOGI(TAG, "[START]\tFreeRTOS: vTaskComunicacionUart started");
15
16        for (;;)
17        {
18            // Esperá notificaciones pero no más de 20 ms (no bloquee una
19            //          ventana entera de 100 Hz)
20            uint32_t flags = 0;
21            (void)xTaskNotifyWait(0, 0xFFFFFFFF, &flags, pdMS_TO_TICKS(50));
22
23            // Cambiar modo solo si llegó el bit de TOGGLE
24            if (flags & UART_NOTIFY_TOGGLE)
25            {
26                modo_transmision = (modo_transmision + 1) % (MODO_MAX + 1);
27                if (DEBUG_MODE)
28                    ESP_LOGI(TAG, "[OK]\tModo de transmisión -> %u", modo_transmision);
29            }
30
31            // Ventana de esoera
32            if (xQueueReceive(xQueueUART, &sDatosSalidaUart, pdMS_TO_TICKS(10)) !=
33                pdTRUE) {
34                continue;
35            }
36
37            // ----- Armado y envío según modo -----
38            switch (modo_transmision)
39            {
40            {
41                case MODO_ADC_DAC:
42                    if (DEBUG_MODE)
43                    {
44                        {
45                            printf("ADC 0:%4d ADC 1:%4d ADC 2:%4d ADC 3:%4d DAC:%3d\n",
46                                sDatosSalidaUart.Valor_ADC[0],
47                                sDatosSalidaUart.Valor_ADC[1],
48                                sDatosSalidaUart.Valor_ADC[2],
49                                sDatosSalidaUart.Valor_ADC[3],
50                                sDatosSalidaUart.valor_DAC);
51                        }
52                    }
53                    n = 0;
54                    mensaje_uart[n++] = HEADER_UART_MODO_ADC_DAC;
55                    mensaje_uart[n++] = lo8(sDatosSalidaUart.Valor_ADC[0]);
56                    mensaje_uart[n++] = hi8(sDatosSalidaUart.Valor_ADC[0]);

```

```
52     mensaje_uart[n++] = lo8(sDatosSalidaUart.Valor_ADC[1]);
53     mensaje_uart[n++] = hi8(sDatosSalidaUart.Valor_ADC[1]);
54     mensaje_uart[n++] = lo8(sDatosSalidaUart.Valor_ADC[2]);
55     mensaje_uart[n++] = hi8(sDatosSalidaUart.Valor_ADC[2]);
56     mensaje_uart[n++] = lo8(sDatosSalidaUart.Valor_ADC[3]);
57     mensaje_uart[n++] = hi8(sDatosSalidaUart.Valor_ADC[3]);
58     mensaje_uart[n++] = sDatosSalidaUart.valor_DAC;
59     break;
60
61 case MODO_MPU:
62     if (DEBUG_MODE)
63     {
64         mpu6050_convert_accel(
65             sDatosSalidaUart.iAceleracion_raw[0],
66             sDatosSalidaUart.iAceleracion_raw[1],
67             sDatosSalidaUart.iAceleracion_raw[2],
68             &fAceleracion_x, &fAceleracion_y, &fAceleracion_z);
69         mpu6050_convert_gyro(
70             sDatosSalidaUart.iGiro_raw[0],
71             sDatosSalidaUart.iGiro_raw[1],
72             sDatosSalidaUart.iGiro_raw[2],
73             &fGiro_x, &fGiro_y, &fGiro_z);
74         roll_pitch_update(fAceleracion_x, fAceleracion_y, fAceleracion_z,
75             fGiro_x, fGiro_y, fGiro_z);
76         printf("A: %.2f %.2f %.2f | G: %.2f %.2f %.2f | Roll: %.2f Pitch:
77             %.2f\n",
78             fAceleracion_x, fAceleracion_y, fAceleracion_z,
79             fGiro_x, fGiro_y, fGiro_z,
80             roll_get(), pitch_get());
81     }
82     n = 0;
83     mensaje_uart[n++] = HEADER_UART_MODO_MPU;
84     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iAceleracion_raw[0])
85     ;
86     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iAceleracion_raw[0])
87     ;
88     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iAceleracion_raw[1])
89     ;
90     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iAceleracion_raw[1])
91     ;
92     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iAceleracion_raw[2])
93     ;
94     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iAceleracion_raw[2])
95     ;
96     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iGiro_raw[0]);
97     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iGiro_raw[0]);
98     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iGiro_raw[1]);
99     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iGiro_raw[1]);
100    mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iGiro_raw[2]);
101    mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iGiro_raw[2]);
102    break;
103
104 case MODO_MPU_ADC_DAC:
105     if (DEBUG_MODE)
106     {
```

```
99         printf("ADC 0:%4d ADC 1:%4d ADC 2:%4d ADC 3:%4d DAC:%3d\n",
100                sDatosSalidaUart.Valor_ADC[0],
101                sDatosSalidaUart.Valor_ADC[1],
102                sDatosSalidaUart.Valor_ADC[2],
103                sDatosSalidaUart.Valor_ADC[3],
104                sDatosSalidaUart.valor_DAC);
105
106         mpu6050_convert_accel(
107             sDatosSalidaUart.iAceleracion_raw[0],
108             sDatosSalidaUart.iAceleracion_raw[1],
109             sDatosSalidaUart.iAceleracion_raw[2],
110             &fAceleracion_x, &fAceleracion_y, &fAceleracion_z);
111         mpu6050_convert_gyro(
112             sDatosSalidaUart.iGiro_raw[0],
113             sDatosSalidaUart.iGiro_raw[1],
114             sDatosSalidaUart.iGiro_raw[2],
115             &fGiro_x, &fGiro_y, &fGiro_z);
116         roll_pitch_update(fAceleracion_x, fAceleracion_y, fAceleracion_z,
117                          fGiro_x, fGiro_y, fGiro_z);
118         printf("A: %.2f %.2f %.2f | G: %.2f %.2f %.2f | Roll: %.2f Pitch:
119                %.2f\n",
120                fAceleracion_x, fAceleracion_y, fAceleracion_z,
121                fGiro_x, fGiro_y, fGiro_z,
122                roll_get(), pitch_get());
123     }
124     n = 0;
125     mensaje_uart[n++] = HEADER_UART_MODAL_MPU_ADC_DAC;
126
127     // ADC + DAC
128     mensaje_uart[n++] = lo8(sDatosSalidaUart.Valor_ADC[0]);
129     mensaje_uart[n++] = hi8(sDatosSalidaUart.Valor_ADC[0]);
130     mensaje_uart[n++] = lo8(sDatosSalidaUart.Valor_ADC[1]);
131     mensaje_uart[n++] = hi8(sDatosSalidaUart.Valor_ADC[1]);
132     mensaje_uart[n++] = lo8(sDatosSalidaUart.Valor_ADC[2]);
133     mensaje_uart[n++] = hi8(sDatosSalidaUart.Valor_ADC[2]);
134     mensaje_uart[n++] = lo8(sDatosSalidaUart.Valor_ADC[3]);
135     mensaje_uart[n++] = hi8(sDatosSalidaUart.Valor_ADC[3]);
136     mensaje_uart[n++] = sDatosSalidaUart.valor_DAC;
137
138     // MPU raw
139     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iAceleracion_raw[0])
140     ;
141     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iAceleracion_raw[0])
142     ;
143     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iAceleracion_raw[1])
144     ;
145     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iAceleracion_raw[1])
146     ;
147     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iAceleracion_raw[2])
148     ;
149     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iAceleracion_raw[2])
150     ;
151     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iGiro_raw[0]);
152     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iGiro_raw[0]);
153     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iGiro_raw[1]);
```



```
146     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iGiro_raw[1]);
147     mensaje_uart[n++] = lo8((uint16_t)sDatosSalidaUart.iGiro_raw[2]);
148     mensaje_uart[n++] = hi8((uint16_t)sDatosSalidaUart.iGiro_raw[2]);
149     break;
150
151     default:
152         if (DEBUG_MODE)
153         {
154             printf("[FALLA] Modo de transmisión desconocido\n");
155             ESP_LOGI(TAG, "[FALLA]\tModo de transmisión desconocido");
156         }
157         n = 0;
158         break;
159     }
160
161     // TX con CRC
162     if (n > 0)
163     {
164         crc = crc_uart(mensaje_uart, n);
165         mensaje_uart[n++] = lo8(crc);
166         mensaje_uart[n++] = hi8(crc);
167
168         uart_write_bytes(UART_PORT_NUM, (const char *)mensaje_uart, n);
169         uart_wait_tx_done(UART_PORT_NUM, pdMS_TO_TICKS(1));
170     }
171
172     // respiro mínimo
173     vTaskDelay(pdMS_TO_TICKS(1));
174 }
175 }
```

Listing 6: Tarea **vTaskComunicacionUart**: Implementación

9 Anexo D: Interfaz de usuario PyQt

```

1  # app_serial_gui.py
2  import sys, time, collections, math
3  from PyQt5 import QtCore, QtWidgets
4  from PyQt5.QtSerialPort import QSerialPort, QSerialPortInfo
5  import pyqtgraph as pg
6  from pyqtgraph.opengl import GLViewWidget, GLLinePlotItem, GLGridItem
7  from PyQt5.QtGui import QVector3D
8  from PyQt5.QtCore import Qt
9  import numpy as np
10
11 # ===== PROTOCOLO =====
12 HEADER_ADC = 0x02      # ADC + DAC
13 HEADER_MPU = 0x04      # Solo IMU
14 HEADER_FULL = 0x07     # Full: ADC + DAC + IMU
15
16 ACK = bytes([0x04])
17 NACK = bytes([0x07])
18
19 PROTO = {
20     HEADER_FULL: {
21         "name": "Full",
22         "frame_len": 24, # 1 hdr + 8 adc + 1 dac + 12 imu + 2 crc
23         "have_adc": True, "have_dac": True, "have_imu": True,
24         "OFF": {
25             # En main.c: se envía LSB,MSB
26             "ADC0": 1, "ADC1": 3, "ADC2": 5, "ADC3": 7,
27             "DAC": 9,
28             "AX": 10, "AY": 12, "AZ": 14, "GX": 16, "GY": 18, "GZ": 20,
29             "CRC": 22
30         }
31     },
32     HEADER_MPU: {
33         "name": "MPU 6050",
34         "frame_len": 15, # 1 hdr + 12 imu + 2 crc
35         "have_adc": False, "have_dac": False, "have_imu": True,
36         "OFF": {
37             "AX": 1, "AY": 3, "AZ": 5, "GX": 7, "GY": 9, "GZ": 11,
38             "CRC": 13
39         }
40     },
41     HEADER_ADC: {
42         "name": "Canales Analógicos",
43         "frame_len": 12, # 1 hdr + 8 adc + 1 dac + 2 crc
44         "have_adc": True, "have_dac": True, "have_imu": False,
45         "OFF": {
46             "ADC0": 1, "ADC1": 3, "ADC2": 5, "ADC3": 7,
47             "DAC": 9,
48             "CRC": 10
49         }
50     }
51 }
52 # =====
53

```

```
54 def crc16_modbus(data: bytes) -> int:
55     crc = 0xFFFF
56     for b in data:
57         crc ^= b
58         for _ in range(8):
59             crc = (crc >> 1) ^ 0xA001 if (crc & 1) else (crc >> 1)
60     return crc & 0xFFFF
61
62 # Escalas MPU6050
63 ACCEL_SCALE = 16384.0
64 GYRO_SCALE  = 131.0
65 GRAVITY     = 9.8
66
67 # Complementario
68 DT         = 0.01
69 ALPHA      = 0.7
70
71 # Escalas visuales
72 MAX_ACC = 15.0 # m/s
73 MAX_GYR = 10.0 # /s
74
75 # Ventana de muestras visible en XY
76 WIN_POINTS = 600
77
78 # ----- Utilidades UI -----
79 def bold_title(text):
80     lab = QtWidgets.QLabel(text)
81     lab.setAlignment(Qt.AlignHCenter)
82     lab.setStyleSheet("font-weight: bold; font-size: 16px;")
83     return lab
84
85 def legend_label(text):
86     lab = QtWidgets.QLabel(text)
87     lab.setAlignment(Qt.AlignHCenter)
88     lab.setStyleSheet("font-weight: bold;")
89     return lab
90
91 def value_label(prefix="", center=False):
92     lab = QtWidgets.QLabel(prefix + " ")
93     lab.setAlignment((Qt.AlignHCenter if center else Qt.AlignLeft) | Qt.
94                     AlignVCenter)
95     lab.setStyleSheet("font-style: italic;")
96     return lab
97
98 def make_frame_with_title(title: str, inner_widget: QtWidgets.QWidget) -> QtWidgets.
99     QWidget:
100     outer = QtWidgets.QWidget()
101     v = QtWidgets.QVBoxLayout(outer); v.setContentsMargins(2,2,2,2); v.setSpacing
102         (6)
103     v.addWidget(bold_title(title))
104     frame = QtWidgets.QFrame(); frame.setFrameShape(QtWidgets.QFrame.Box); frame.
105         setLineWidth(1)
106     fv = QtWidgets.QVBoxLayout(frame); fv.setContentsMargins(8,8,8,8); fv.
107         setSpacing(6)
108     fv.addWidget(inner_widget)
```

```

104     v.addWidget(frame)
105     return outer
106
107 # ----- VBox con modo seguir -----
108 class FollowVBox(pg.ViewBox):
109     """Permite zoom/pan en X. Si el usuario interactúa, desactiva 'seguir'.
110     Cuando 'seguir' está activo, el gráfico usa ventana deslizante."""
111     def __init__(self, *args, **kwargs):
112         super().__init__(*args, enableMenu=True, **kwargs)
113         self.follow = True
114         self.setMouseEnabled(x=True, y=False)
115
116     def wheelEvent(self, ev):
117         self.follow = False
118         super().wheelEvent(ev)
119
120     def mouseDragEvent(self, ev, axis=None):
121         if ev.button() == Qt.LeftButton:
122             self.follow = False
123         super().mouseDragEvent(ev, axis=axis)
124
125 # ----- Flecha 3D -----
126 class Arrow3D(GLLinePlotItem):
127     def __init__(self, color=(1,1,1,1)):
128         super().__init__(mode='lines', antialias=True)
129         self.color = color
130         self.update_vec(0,0,0)
131     def update_vec(self, x, y, z):
132         tip = np.array([float(x), float(y), float(z)], dtype=float)
133         L = max(1e-6, float((tip**2).sum())**0.5)
134         s = 0.06 * L
135         a = np.array([1.0,0.0,0.0])
136         if L > 0 and abs((a*tip).sum()/L) > 0.9:
137             a = np.array([0.0,1.0,0.0])
138         b = np.cross(tip, a)
139         if (b**2).sum() > 0: b = b/((b**2).sum())**0.5
140         a = np.cross(b, tip)
141         if (a**2).sum() > 0: a = a/((a**2).sum())**0.5
142         base = np.zeros(3); neck = tip - 0.1*tip
143         p1 = neck + a*s; p2 = neck - a*s; p3 = neck + b*s; p4 = neck - b*s
144         pts = np.array([ base, tip, tip, p1, tip, p2, tip, p3, tip, p4 ], dtype=
            float)
145         self.setData(pos=pts, color=self.color, width=2.0)
146
147 # ----- RPY -> Vector unitario para Roll, Pitch, Yaw -----
148 def rpy_to_unit_vector(roll_deg: float, pitch_deg: float, yaw_deg: float):
149     r, p, y = map(math.radians, (roll_deg, pitch_deg, yaw_deg))
150     cr, sr = math.cos(r), math.sin(r)
151     cp, sp = math.cos(p), math.sin(p)
152     cy, sy = math.cos(y), math.sin(y)
153
154     vx = cy*sp*cr + sy*sr
155     vy = sy*sp*cr - cy*sr
156     vz = cp*cr
157     n = math.sqrt(vx*vx + vy*vy + vz*vz)

```

```
158     return (vx/n, vy/n, vz/n) if n else (0.0, 0.0, 1.0)
159
160 # ----- Ventana principal -----
161 class MainWindow(QMainWindow):
162     def __init__(self):
163         super().__init__()
164         self.setWindowTitle("TD3 TP3 : Monitor inercial con ESP32")
165         cw = QtWidgets.QWidget(); self.setCentralWidget(cw)
166
167     # ----- Barra superior -----
168     self.portBox = QtWidgets.QComboBox()
169     self.baudBox = QtWidgets.QComboBox()
170     self.btnRefresh = QtWidgets.QToolButton(text="  ")
171     self.btnConn = QtWidgets.QPushButton("Conectar")
172     self.modeBox = QtWidgets.QLineEdit(); self.modeBox.setReadOnly(True); self.
        modeBox.setPlaceholderText("Modo:  ")
173
174     for w in (self.portBox, self.baudBox, self.modeBox, self.btnConn, self.
        btnRefresh):
175         w.setMinimumHeight(28)
176         w.setStyleSheet("font-size: 11pt;")
177
178     labPuerto = QtWidgets.QLabel("Puerto"); labPuerto.setStyleSheet("font-size:
        11pt; font-weight: bold;")
179     labBaud = QtWidgets.QLabel("Baud Rate"); labBaud.setStyleSheet("font-size
        : 11pt; font-weight: bold;")
180     labModo = QtWidgets.QLabel("Modo"); labModo.setStyleSheet("font-size: 11
        pt; font-weight: bold;")
181
182     self.btnRefresh.clicked.connect(self.refresh_ports)
183     self.btnConn.clicked.connect(self.toggle_port)
184     self.refresh_ports()
185     self.baudBox.addItem(["9600", "19200", "38400", "57600", "115200", "230400", "
        460800", "921600"])
186
187     # Botón para reactivar seguimiento en eje X
188     self.btnFollow = QtWidgets.QPushButton("Seguir X"); self.btnFollow.
        setCheckable(True); self.btnFollow.setChecked(True)
189     self.btnFollow.setToolTip("Cuando está activo, el gráfico avanza automá
        ticamente.")
190
191     top = QtWidgets.QHBoxLayout()
192     top.addWidget(labPuerto); top.addWidget(self.portBox, 1); top.addWidget(
        self.btnRefresh)
193     top.addSpacing(12); top.addWidget(labBaud); top.addWidget(self.baudBox);
        top.addWidget(self.btnConn)
194     top.addStretch(1); top.addWidget(labModo); top.addWidget(self.modeBox); top
        .addSpacing(12); top.addWidget(self.btnFollow)
195
196     # ----- Plots ADC/DAC -----
197     pg.setConfigOptions(antialias=True)
198
199     # ADC
200     self.vbADC = FollowViewBox()
201     self.plotADC = pg.PlotWidget(viewBox=self.vbADC)
```

```

202     self.plotADC.showGrid(x=True, y=True)
203     self.plotADC.setLabel('bottom', 'Muestras', units='')
204     self.plotADC.setYRange(0, 3.3)
205     self.plotADC.setTitle("Entradas Analógicas")
206     self.plotADC.setLabel('left', 'Voltaje', units='V')
207     self.curves_adc = [
208         self.plotADC.plot(pen=pg.mkPen('y', width=1.5)), # ADC0      Amarillo
209         self.plotADC.plot(pen=pg.mkPen('g', width=1.5)), # ADC1      Verde
210         self.plotADC.plot(pen=pg.mkPen('r', width=1.5)), # ADC2      Rojo
211         self.plotADC.plot(pen=pg.mkPen('c', width=1.5))  # ADC3      Cian
212     ]
213
214     # Leyendas de curvas en la esquina superior derecha
215     self.legendADC = self.plotADC.addLegend(offset=(-10, 10)) # dentro del á
216         rea del gráfico
217     self.legendADC.setBrush(pg.mkBrush(0, 0, 0, 150)) # fondo
218         semitransparente
219     self.legendADC.anchor((1, 0), (1, 0), offset=(-10, 10)) # esquina
220         superior derecha
221
222     # Asociación de nombres y curvas
223     self.legendADC.addItem(self.curves_adc[0], "CH1")
224     self.legendADC.addItem(self.curves_adc[1], "CH2")
225     self.legendADC.addItem(self.curves_adc[2], "CH3")
226     self.legendADC.addItem(self.curves_adc[3], "CH4")
227
228     # DAC
229     self.vbDAC = FollowViewBox()
230     self.plotDAC = pg.PlotWidget(viewBox=self.vbDAC)
231     self.plotDAC.showGrid(x=True, y=True)
232     self.plotDAC.setLabel('bottom', 'Muestras', units='')
233     self.plotDAC.setYRange(0, 3.3)
234     self.plotDAC.setTitle("Salida Analógica")
235     self.plotDAC.setLabel('left', 'Voltaje', units='V')
236     self.curve_dac = self.plotDAC.plot(pen='y')
237
238     self.max_points = 50000
239     self.buf_adc = [collections.deque(maxlen=self.max_points) for _ in range(4)]
240     self.buf_dac = collections.deque(maxlen=self.max_points)
241
242     self.lab_adc_vals = [value_label(f"CH{i+1}: ", center=True) for i in range
243         (4)]
244     self.lab_dac_val = value_label("DAC: ", center=True)
245
246     # XY dentro del marco
247     analog_grid = QtWidgets.QGridLayout()
248     analog_grid.setHorizontalSpacing(18); analog_grid.setVerticalSpacing(10)
249     analog_grid.addWidget(self.plotADC, 0, 0)
250     analog_grid.addWidget(self.plotDAC, 0, 1)
251
252     adc_vals_row = QtWidgets.QHBoxLayout(); adc_vals_row.setSpacing(20);
253     adc_vals_row.setAlignment(Qt.AlignHCenter)
254     for lab in self.lab_adc_vals: adc_vals_row.addWidget(lab)

```

```

251     dac_vals_row = QtWidgets.QHBoxLayout(); dac_vals_row.setSpacing(20);
252         dac_vals_row.setAlignment(Qt.AlignHCenter)
253     dac_vals_row.addWidget(self.lab_dac_val)
254
255     analog_grid.addLayout(adc_vals_row, 1, 0)
256     analog_grid.addLayout(dac_vals_row, 1, 1)
257
258     analog_inner = QtWidgets.QWidget(); analog_inner.setLayout(analog_grid)
259     analog_group = make_frame_with_title("Canales Analógicos", analog_inner)
260
261     # ----- Vistas 3D -----
262     self.viewAcc = self._make_3d_view()
263     self.viewGyr = self._make_3d_view()
264     self.viewRPY = self._make_3d_view()
265     self.viewAcc.setMinimumHeight(360)
266     self.viewGyr.setMinimumHeight(360)
267     self.viewRPY.setMinimumHeight(360)
268     self.viewAcc.setSizePolicy(QtWidgets.QSizePolicy.Expanding, QtWidgets.
269         QSizePolicy.Expanding)
270     self.viewGyr.setSizePolicy(QtWidgets.QSizePolicy.Expanding, QtWidgets.
271         QSizePolicy.Expanding)
272     self.viewRPY.setSizePolicy(QtWidgets.QSizePolicy.Expanding, QtWidgets.
273         QSizePolicy.Expanding)
274
275     self.arrowAcc = Arrow3D(color=(0,1,0,1))
276     self.arrowGyr = Arrow3D(color=(1,0,0,1))
277     self.arrowRPY = Arrow3D(color=(0.2,0.6,1.0,1))
278     self.viewAcc.addItem(self.arrowAcc)
279     self.viewGyr.addItem(self.arrowGyr)
280     self.viewRPY.addItem(self.arrowRPY)
281
282     self.lab_acc_xyz = [value_label("x : ", True), value_label("y : ", True),
283         value_label("z : ", True)]
284     self.lab_gyr_xyz = [value_label("x : ", True), value_label("y : ", True),
285         value_label("z : ", True)]
286     self.lab_rpy_xyz = [value_label("Roll : ", True), value_label("Pitch : ",
287         True), value_label("Yaw : ", True)]
288
289     def pack_3d(title, glw, labs):
290         w = QtWidgets.QWidget()
291         v = QtWidgets.QVBoxLayout(w); v.setContentsMargins(0,0,0,0); v.
292             setSpacing(6)
293         v.addWidget(legend_label(title))
294         v.addWidget(glw, 1) # ocupa el máximo de ventana
295         row = QtWidgets.QHBoxLayout(); row.setSpacing(20); row.setAlignment(Qt.
296             AlignHCenter)
297         for l in labs: row.addWidget(l)
298         v.addLayout(row)
299         return w
300
301     accW = pack_3d("Aceleración [m/s2]", self.viewAcc, self.lab_acc_xyz)
302     gyrW = pack_3d("Velocidad angular [ /s]", self.viewGyr, self.lab_gyr_xyz)
303     rpyW = pack_3d("Roll, Pitch, Yaw [ ]", self.viewRPY, self.lab_rpy_xyz)
304
305     inertial_row = QtWidgets.QHBoxLayout()

```

```
297     inertial_row.setSpacing(12)
298     inertial_row.addWidget(accW); inertial_row.setStretchFactor(accW, 1)
299     inertial_row.addWidget(gyrW); inertial_row.setStretchFactor(gyrW, 1)
300     inertial_row.addWidget(rpyW); inertial_row.setStretchFactor(rpyW, 1)
301
302     inertial_inner = QtWidgets.QWidget(); inertial_inner.setLayout(inertial_row
303     )
304     inertial_group = make_frame_with_title("Datos de movimiento inercial",
305     inertial_inner)
306
307     # ----- Registro -----
308     self.logEdit = QtWidgets.QPlainTextEdit(); self.logEdit.setReadOnly(True)
309
310     # ----- Layout principal -----
311     lay = QtWidgets.QVBoxLayout(cw)
312     lay.addLayout(top)
313     lay.addWidget(analog_group)
314     lay.addWidget(inertial_group, 1) # prioridad vertical a los 3D
315     lay.addWidget(QtWidgets.QLabel("Registro"))
316     lay.addWidget(self.logEdit, 1)
317
318     # ----- Serial -----
319     self.serial = QSerialPort(readyRead=self.on_ready_read)
320     self.rxbuf = bytearray()
321     self.current_mode = None
322     self._roll = 0.0; self._pitch = 0.0; self._yaw = 0.0
323
324     # Timer
325     self.timer = QtCore.QTimer(interval=33, timeout=self.refresh_plots)
326     self.timer.start()
327
328     # Señal botón seguir
329     self.btnFollow.toggled.connect(self._on_follow_toggled)
330
331     # ----- 3D view base -----
332     def _make_3d_view(self):
333         v = GLViewWidget()
334         v.opts['distance'] = 120
335         gxy = GLGridItem(); gxy.setSize(60,60); gxy.setSpacing(10,10); v.addItem(
336             gxy)
337         gxz = GLGridItem(); gxz.rotate(90,1,0,0); gxz.translate(0,0,-50); gxz.
338             setSize(60,60); gxz.setSpacing(10,10); v.addItem(gxz)
339         gyz = GLGridItem(); gyz.rotate(90,0,1,0); gyz.translate(-50,0,0); gyz.
340             setSize(60,60); gyz.setSpacing(10,10); v.addItem(gyz)
341         v.opts['center'] = QVector3D(25,25,25)
342         return v
343
344     # ----- puertos -----
345     def refresh_ports(self):
346         cur = self.portBox.currentText()
347         self.portBox.clear()
348         for p in QSerialPortInfo.availablePorts():
349             self.portBox.addItem(p.portName())
350         if cur:
351             idx = self.portBox.findText(cur)
```



```
347         if idx >= 0: self.portBox.setCurrentIndex(idx)
348
349     def toggle_port(self):
350         if self.serial.isOpen():
351             self.serial.close(); self.btnConn.setText("Conectar")
352             return
353         port = self.portBox.currentText()
354         if not port:
355             QtWidgets.QMessageBox.warning(self, "Puerto", "No hay puerto seleccionado
356             ."); return
357         self.serial.setPortName(port)
358         self.serial.setBaudRate(int(self.baudBox.currentText()))
359         self.serial.setDataBits(QSerialPort.Data8)
360         self.serial.setParity(QSerialPort.NoParity)
361         self.serial.setStopBits(QSerialPort.OneStop)
362         self.serial.setFlowControl(QSerialPort.NoFlowControl)
363         if not self.serial.open(QtCore.QIODevice.ReadWrite):
364             QtWidgets.QMessageBox.critical(self, "Error", "No se pudo abrir el puerto
365             ."); return
366         self.btnConn.setText("Desconectar")
367
368     # Reset buffers y vista
369     for i in range(4): self.buf_adc[i].clear()
370     self.buf_dac.clear()
371     self.vbADC.follow = True
372     self.vbDAC.follow = True
373     self.btnFollow.setChecked(True)
374     self.plotADC.enableAutoRange(x=False, y=False)
375     self.plotDAC.enableAutoRange(x=False, y=False)
376     self.plotADC.setXRange(0, WIN_POINTS, padding=0)
377     self.plotDAC.setXRange(0, WIN_POINTS, padding=0)
378
379     def _on_follow_toggled(self, checked: bool):
380         self.vbADC.follow = checked
381         self.vbDAC.follow = checked
382         if checked:
383             # Reenganchar muestra actual en X
384             nA = max(len(b) for b in self.buf_adc) if self.buf_adc else 0
385             nD = len(self.buf_dac)
386             if nA <= WIN_POINTS:
387                 self.plotADC.setXRange(0, WIN_POINTS, padding=0)
388             else:
389                 self.plotADC.setXRange(nA - WIN_POINTS, nA, padding=0)
390             if nD <= WIN_POINTS:
391                 self.plotDAC.setXRange(0, WIN_POINTS, padding=0)
392             else:
393                 self.plotDAC.setXRange(nD - WIN_POINTS, nD, padding=0)
394
395     # ——— lectura serie ———
396     def on_ready_read(self):
397         self.rxbuf += self.serial.readAll().data()
398         while True:
399             if not self.rxbuf: return
400             hdr = self.rxbuf[0]
401             if hdr not in PROT0:
```

```

400         pos = next((i for i,b in enumerate(self.rxbuf) if b in PROTO), -1)
401         if pos < 0: self.rxbuf.clear(); return
402         if pos > 0: del self.rxbuf[:pos]
403         hdr = self.rxbuf[0]
404         cfg = PROTO[hdr]
405         if len(self.rxbuf) < cfg["frame_len"]: return
406         frame = bytes(self.rxbuf[:cfg["frame_len"]])
407         del self.rxbuf[:cfg["frame_len"]]
408         self.process_frame_by_mode(hdr, frame)
409
410     @staticmethod
411     def _int16_from_bytes(lsb, msb):
412         u = (msb << 8) | lsb
413         return u - 65536 if u >= 32768 else u
414
415     def process_frame_by_mode(self, hdr: int, frame: bytes):
416         cfg = PROTO[hdr]
417
418         if self.current_mode != hdr:
419             self.current_mode = hdr
420             self.modeBox.setText(cfg["name"])
421
422         # CRC bilateral (acepta H,L o L,H)
423         crc_off = cfg["OFF"]["CRC"]
424         recv_h = frame[crc_off]; recv_l = frame[crc_off+1]
425         calc = crc16_modbus(frame[:crc_off])
426         ok = ((recv_h == ((calc>>8)&0xFF) and recv_l == (calc&0xFF)) or
427              (recv_l == ((calc>>8)&0xFF) and recv_h == (calc&0xFF)))
428         self.serial.write(ACK if ok else NACK)
429         if not ok: return
430
431         def get16s(off): return self._int16_from_bytes(frame[off], frame[off+1])
432         def u16_le(off): return ((frame[off+1] << 8) | frame[off]) & 0xFFFF
433
434         # Variables para registro
435         adc0 = adc1 = adc2 = adc3 = dac = None
436         Ax_raw = Ay_raw = Az_raw = Gx_raw = Gy_raw = Gz_raw = None
437         Ax = Ay = Az = Gx = Gy = Gz = None
438
439         # ADC/DAC
440         if cfg["have_dac"]:
441             dac = frame[cfg["OFF"]["DAC"]]
442             self.buf_dac.append(dac)
443             self.lab_dac_val.setText(f"DAC: {dac}")
444         else:
445             # Empujar "muestra vacía" para que X avance sin dibujar cuando se
446             # cambia de modo
447             self.buf_dac.append(np.nan)
448
449         if cfg["have_adc"]:
450             adc0 = u16_le(cfg["OFF"]["ADC0"]); adc1 = u16_le(cfg["OFF"]["ADC1"])
451             adc2 = u16_le(cfg["OFF"]["ADC2"]); adc3 = u16_le(cfg["OFF"]["ADC3"])
452             self.buf_adc[0].append(adc0); self.buf_adc[1].append(adc1)
453             self.buf_adc[2].append(adc2); self.buf_adc[3].append(adc3)
454             self.lab_adc_vals[0].setText(f"CH1: {adc0}")

```

```

454         self.lab_adc_vals[1].setText(f"CH2: {adc1}")
455         self.lab_adc_vals[2].setText(f"CH3: {adc2}")
456         self.lab_adc_vals[3].setText(f"CH4: {adc3}")
457     else:
458         for i in range(4):
459             self.buf_adc[i].append(np.nan)
460
461     # IMU
462     if cfg["have_imu"]:
463         Ax_raw = get16s(cfg["OFF"]["AX"]); Ay_raw = get16s(cfg["OFF"]["AY"]);
464         Az_raw = get16s(cfg["OFF"]["AZ"]);
465         Gx_raw = get16s(cfg["OFF"]["GX"]); Gy_raw = get16s(cfg["OFF"]["GY"]);
466         Gz_raw = get16s(cfg["OFF"]["GZ"]);
467
468         Ax = (Ax_raw / ACCEL_SCALE) * GRAVITY
469         Ay = (Ay_raw / ACCEL_SCALE) * GRAVITY
470         Az = (Az_raw / ACCEL_SCALE) * GRAVITY
471         Gx = (Gx_raw / GYRO_SCALE) # /s
472         Gy = (Gy_raw / GYRO_SCALE)
473         Gz = (Gz_raw / GYRO_SCALE)
474
475         # Complementario para RPY (en grados)
476         accel_roll = math.degrees(math.atan2(Ay, Az))
477         accel_pitch = math.degrees(math.atan2(-Ax, math.sqrt(Ay*Ay + Az*Az)))
478         self._roll = ALPHA*(self._roll + Gx*DT) + (1-ALPHA)*accel_roll
479         self._pitch = ALPHA*(self._pitch + Gy*DT) + (1-ALPHA)*accel_pitch
480         self._yaw = self._yaw + Gz*DT
481
482         # Vector unitario para visualizar RPY (flecha longitud 50)
483         ux, uy, uz = rpy_to_unit_vector(self._roll, self._pitch, self._yaw)
484         self.arrowRPY.update_vec(50.0*ux, 50.0*uy, 50.0*uz)
485
486         # Aceleración / Giro
487         clamp = lambda v: max(-50, min(50, v))
488         self.arrowAcc.update_vec(
489             clamp(Ax * (50.0 / MAX_ACC)),
490             clamp(Ay * (50.0 / MAX_ACC)),
491             clamp(Az * (50.0 / MAX_ACC))
492         )
493         self.arrowGyr.update_vec(
494             clamp(Gx * (50.0 / MAX_GYR)),
495             clamp(Gy * (50.0 / MAX_GYR)),
496             clamp(Gz * (50.0 / MAX_GYR))
497         )
498
499         # Etiquetas
500         self.lab_acc_xyz[0].setText(f"x : {Ax:+.2f}")
501         self.lab_acc_xyz[1].setText(f"y : {Ay:+.2f}")
502         self.lab_acc_xyz[2].setText(f"z : {Az:+.2f}")
503         self.lab_gyr_xyz[0].setText(f"x : {Gx:+.2f}")
504         self.lab_gyr_xyz[1].setText(f"y : {Gy:+.2f}")
505         self.lab_gyr_xyz[2].setText(f"z : {Gz:+.2f}")
506
507     def wrap180(a): return (a + 180.0) % 360.0 - 180.0
508     self.lab_rpy_xyz[0].setText(f"Roll : {wrap180(self._roll):+.1f} ")

```

```

507         self.lab_rpy_xyz[1].setText(f"Pitch : {wrap180(self._pitch):+.1f} ")
508         self.lab_rpy_xyz[2].setText(f"Yaw : {wrap180(self._yaw):+.1f} ")
509
510     # ----- Registro detallado -----
511     ts = time.strftime("%H:%M:%S")
512     parts = [f"{ts} - "]
513     if cfg["have_adc"]:
514         parts.append(f"ADC0:{adc0} ADC1:{adc1} ADC2:{adc2} ADC3:{adc3} ")
515     else:
516         parts.append("ADC0:      ADC1:      ADC2:      ADC3:      ")
517     parts.append(f"DAC:{dac if dac is not None else ' ' } ")
518
519     if cfg["have_imu"]:
520         parts.append(f"Ax_raw:{Ax_raw} Ay_raw:{Ay_raw} Az_raw:{Az_raw} "
521                     f"Gx_raw:{Gx_raw} Gy_raw:{Gy_raw} Gz_raw:{Gz_raw} | ")
522         parts.append(f"A[g]:{(Ax/GRAVITY):+.2f} {(Ay/GRAVITY):+.2f} {(Az/
523                     GRAVITY):+.2f} "
524                     f"G[ /s]:{Gx:+.2f} {Gy:+.2f} {Gz:+.2f} | ")
525         parts.append(f"Roll:{self._roll:+.1f} Pitch:{self._pitch:+.1f} Yaw
526                     :{self._yaw:+.1f} ")
527     else:
528         parts.append("Ax_raw:      Ay_raw:      Az_raw:      Gx_raw:      Gy_raw:
529                     Gz_raw:      ")
530
531     self.logEdit.appendPlainText("".join(parts))
532
533     # ----- refresco XY -----
534     def refresh_plots(self):
535         # ADC
536         for i, c in enumerate(self.curves_adc):
537             n = len(self.buf_adc[i])
538             if n:
539                 x = np.arange(n, dtype=float)
540                 y = np.fromiter((v * 3.3 / 4095.0 for v in self.buf_adc[i]), dtype=
541                               float, count=n)
542                 c.setData(x, y)
543
544         # DAC
545         nd = len(self.buf_dac)
546         if nd:
547             xd = np.arange(nd, dtype=float)
548             yd = np.fromiter((v * 3.3 / 255.0 for v in self.buf_dac), dtype=float,
549                             count=nd)
550             self.curve_dac.setData(xd, yd)
551
552         # Rango X: sólo si 'seguir' está activo
553         if self.vbADC.follow:
554             nA = max((len(b) for b in self.buf_adc), default=0)
555             if nA <= WIN_POINTS:
556                 self.plotADC.setXRange(0, WIN_POINTS, padding=0)
557             else:
558                 self.plotADC.setXRange(nA - WIN_POINTS, nA, padding=0)
559         if self.vbDAC.follow:
560             nD = len(self.buf_dac)
561             if nD <= WIN_POINTS:
562                 self.plotDAC.setXRange(0, WIN_POINTS, padding=0)

```

```
557         else:
558             self.plotDAC.setXRange(nD - WIN_POINTS, nD, padding=0)
559
560 if __name__ == "__main__":
561     app = QtWidgets.QApplication(sys.argv)
562     w = MainWindow(); w.resize(1280, 900); w.show()
563     sys.exit(app.exec_())
```

Listing 7: Título opcional